

Gemini code(1st)

Claude code(2nd)

Accessed via gemini and claude on my laptop

Prompt used:

Write a C program that prioritizes emails for a busy company CEO. You must implement a MaxHeap using a list-based implementation to serve as a priority queue. Do not use any pre-existing heap modules; all MaxHeap and queue functions must be created from scratch.

The program needs to process a test file containing the following commands:

1. EMAIL , , Adds an email to the queue based on priority. 'EMAIL' is followed by a space, and the remaining fields are comma-delimited.

- The sender categories, in strict order of priority (highest to lowest), are: Boss, Subordinate, Peer, ImportantPerson, Other Person.
- **Tie-breaker:** If there are multiple emails in the queue from the same sender category, the *newest* email (by date) must be prioritized and read first.
- Assume the subject line may contain spaces but no commas, and the date is always formatted as MM-DD-YYYY.

2. NEXT Displays the highest priority email currently in the queue. The output must be formatted exactly as follows: Next email: Sender: [sender] Subject: [subject] Date: [date]

3. READ Removes the highest priority email from the queue (simulating that the CEO has read and dealt with it).

4. COUNT Displays the number of unread emails in the format: "There are [X] emails to read."

Your code must be robust enough to handle varying orders and quantities of commands. For example, it must cleanly handle calling NEXT or READ when there are zero emails in the queue, calling NEXT multiple times without an intervening READ (the output should remain the same), and calling READ multiple times without an intervening NEXT (silently deleting the highest priority emails).

Gemini code: `#include <stdio.h>`

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// --- Data Structures ---
```

```
typedef struct {
```

```
    char sender[64];
```

```
    char subject[128];
```

```
    char date_str[16];
```

```

        int priority_score;
        int date_score;
    } Email;

typedef struct {
    Email *array;
    int size;
    int capacity;
} MaxHeap;

// --- Helper Functions ---

// Assigns an integer value to the sender for priority comparison
int get_sender_priority(const char* sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "Other Person") == 0) return 1;
    return 0; // Fallback for unknown senders
}

// Converts a MM-DD-YYYY date into an integer YYYYMMDD for easy numerical
comparison
int parse_date(const char* date) {
    int m, d, y;
    if (sscanf(date, "%d-%d-%d", &m, &d, &y) == 3) {
        return y * 10000 + m * 100 + d;
    }
    return 0;
}

// Compares two emails. Returns >0 if a has higher priority, <0 if b has
higher, 0 if equal
int compare_emails(Email a, Email b) {
    if (a.priority_score != b.priority_score) {
        return a.priority_score - b.priority_score;
    }
    // Tie-breaker: Newer date gets read first (larger integer value)

```

```

        return a.date_score - b.date_score;
    }

// --- MaxHeap Implementations ---

// Initializes the heap
MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->array = (Email*)malloc(sizeof(Email) * initial_capacity);
    heap->size = 0;
    heap->capacity = initial_capacity;
    return heap;
}

// Swaps two elements in the heap
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

// Bubbles up the element at index i to maintain the max-heap property
void heapify_up(MaxHeap *heap, int i) {
    int parent = (i - 1) / 2;
    // Keep bubbling up as long as we aren't the root and our priority is
    // greater than the parent's
    while (i > 0 && compare_emails(heap->array[i], heap->array[parent]) >
0) {
        swap(&heap->array[i], &heap->array[parent]);
        i = parent;
        parent = (i - 1) / 2;
    }
}

// Bubbles down the element at index i to maintain the max-heap property
void heapify_down(MaxHeap *heap, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

```

```

        if (left < heap->size && compare_emails(heap->array[left],
heap->array[largest]) > 0) {
            largest = left;
        }
        if (right < heap->size && compare_emails(heap->array[right],
heap->array[largest]) > 0) {
            largest = right;
        }

        if (largest != i) {
            swap(&heap->array[i], &heap->array[largest]);
            heapify_down(heap, largest);
        }
    }
}

// Inserts a new email into the heap
void insert_email(MaxHeap *heap, Email email) {
    // Resize array if it hits capacity
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->array = (Email*)realloc(heap->array, sizeof(Email) *
heap->capacity);
    }

    // Insert at the end and heapify up
    heap->array[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

// Removes and returns the highest priority email
// (If size == 0, we simply ignore it since the queue is empty)
Email read_email(MaxHeap *heap) {
    Email removed = {0};

    if (heap->size <= 0) {
        return removed;
    }
}

```

```

        removed = heap->array[0];

        // Swap the root with the last element
        heap->array[0] = heap->array[heap->size - 1];
        heap->size--;

        // Restore the max-heap property starting from the new root
        if (heap->size > 0) {
            heapify_down(heap, 0);
        }

        return removed;
    }

// Displays the highest priority email without removing it
void next_email(MaxHeap *heap) {
    if (heap->size <= 0) {
        // Standard clean handling for empty queue - silent return or
        // user-friendly message
        // Left silent/no-op for strict adherence to formatting unless
        // there are emails.
        return;
    }
    printf("Next email:\n");
    printf("Sender: %s\n", heap->array[0].sender);
    printf("Subject: %s\n", heap->array[0].subject);
    printf("Date: %s\n", heap->array[0].date_str);
}

// --- Main Execution ---

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <commands_file.txt>\n", argv[0]);
        return 1;
    }

    FILE *file = fopen(argv[1], "r");
    if (!file) {

```

```

    printf("Error opening file: %s\n", argv[1]);
    return 1;
}

MaxHeap *queue = create_heap(10);
char line[256];

while (fgets(line, sizeof(line), file)) {
    // Strip out trailing carriage return or newline
    line[strcspn(line, "\r\n")] = '\0';

    if (strncmp(line, "EMAIL ", 6) == 0) {
        char *rest = line + 6; // Move pointer past "EMAIL "
        char *sender = strtok(rest, ",");
        char *subject = strtok(NULL, ",");
        char *date = strtok(NULL, ",");

        if (sender && subject && date) {
            Email e;
            strcpy(e.sender, sender);
            strcpy(e.subject, subject);
            strcpy(e.date_str, date);

            e.priority_score = get_sender_priority(sender);
            e.date_score = parse_date(date);

            insert_email(queue, e);
        }
    }
    else if (strcmp(line, "NEXT") == 0) {
        next_email(queue);
    }
    else if (strcmp(line, "READ") == 0) {
        read_email(queue);
    }
    else if (strcmp(line, "COUNT") == 0) {
        printf("There are %d emails to read.\n", queue->size);
    }
}

```

```

    // Clean up memory
    fclose(file);
    free(queue->array);
    free(queue);

    return 0;
}

```

Claude Code: /* =====

```

* email_priority.c
*
* Priority queue for a CEO's inbox, backed by a hand-written,
* array (list) based MaxHeap. No library heap/PQ code is used;
* every heap operation (push-up, sift-down, resize) is written
* from scratch below.
*
* Priority rules:
*   1. Sender category, highest first:
*       Boss > Subordinate > Peer > ImportantPerson > Other Person
*   2. Within the same category, the newest email (by date) wins.
*
* Commands (read one per line from a file or stdin):
*   EMAIL <sender>,<subject>,<date>    date is MM-DD-YYYY
*   NEXT                               show highest-priority email
*   READ                               remove highest-priority email
*   COUNT                              show number of unread emails
*   ===== */

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

```

```

/* ----- tunables / constants ----- */
#define INITIAL_CAPACITY 16
#define MAX_LINE_LEN    1024
#define MAX_SENDER_LEN  128

```

```

#define MAX_SUBJECT_LEN 512
#define MAX_DATE_LEN 16

/* ----- Email record ----- */
typedef struct {
    char sender[MAX_SENDER_LEN];
    char subject[MAX_SUBJECT_LEN];
    char date[MAX_DATE_LEN]; /* stored as given, MM-DD-YYYY */
    int categoryRank; /* higher = more important */
    int dateValue; /* YYYYMMDD, higher = newer */
} Email;

/* ----- List-based MaxHeap ----- */
typedef struct {
    Email *data; /* dynamically allocated array (the "list") */
    int size; /* number of elements currently stored */
    int capacity; /* allocated slots */
} MaxHeap;

/* =====
 * Category ranking
 *
 * Boss > Subordinate > Peer > ImportantPerson > Other Person
 * We give Boss the highest integer so "bigger number wins" holds
 * consistently for both category and date comparisons.
 * ===== */
static int categoryRank(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "Other Person") == 0) return 1;
    /* Unknown/unrecognized sender category: treat as lowest priority
       rather than crashing or rejecting the email. */
    return 0;
}

/* Convert "MM-DD-YYYY" into an integer YYYYMMDD so that a plain
 * numeric comparison tells us which date is newer (bigger = newer). */
static int dateToInt(const char *date) {

```

```

    int mm = 0, dd = 0, yyyy = 0;
    if (sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy) != 3) {
        return 0; /* malformed date: push to the back, don't crash */
    }
    return yyyy * 10000 + mm * 100 + dd;
}

/* =====
 * MaxHeap: construction / destruction
 * ===== */
static void heapInit(MaxHeap *h) {
    h->capacity = INITIAL_CAPACITY;
    h->size = 0;
    h->data = (Email *)malloc(sizeof(Email) * h->capacity);
    if (!h->data) {
        fprintf(stderr, "Fatal: out of memory initializing heap.\n");
        exit(1);
    }
}

static void heapFree(MaxHeap *h) {
    free(h->data);
    h->data = NULL;
    h->size = 0;
    h->capacity = 0;
}

static void heapGrowIfNeeded(MaxHeap *h) {
    if (h->size >= h->capacity) {
        int newCapacity = h->capacity * 2;
        Email *newData = (Email *)realloc(h->data, sizeof(Email) *
newCapacity);
        if (!newData) {
            fprintf(stderr, "Fatal: out of memory growing heap.\n");
            exit(1);
        }
        h->data = newData;
        h->capacity = newCapacity;
    }
}

```

```

/* =====
 * Ordering: returns 1 if email a has strictly higher priority
 * than email b, else 0.
 * ===== */
static int hasHigherPriority(const Email *a, const Email *b) {
    if (a->categoryRank != b->categoryRank) {
        return a->categoryRank > b->categoryRank;
    }
    /* same category: newer date wins */
    return a->dateValue > b->dateValue;
}

static void swapEmails(Email *a, Email *b) {
    Email tmp = *a;
    *a = *b;
    *b = tmp;
}

/* ----- Sift the last element up until heap order holds -----
*/
static void siftUp(MaxHeap *h, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (hasHigherPriority(&h->data[index], &h->data[parent])) {
            swapEmails(&h->data[index], &h->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

/* ----- Sift an element down until heap order holds ----- */
static void siftDown(MaxHeap *h, int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

```

```

        if (left < h->size && hasHigherPriority(&h->data[left],
&h->data[largest])) {
            largest = left;
        }
        if (right < h->size && hasHigherPriority(&h->data[right],
&h->data[largest])) {
            largest = right;
        }
        if (largest == index) {
            break;
        }
        swapEmails(&h->data[index], &h->data[largest]);
        index = largest;
    }
}

/* ----- Public heap operations ----- */
static void heapInsert(MaxHeap *h, Email e) {
    heapGrowIfNeeded(h);
    h->data[h->size] = e;
    siftUp(h, h->size);
    h->size++;
}

/* Returns 1 and copies the max element into *out without removing it.
 * Returns 0 if the heap is empty. */
static int heapPeek(MaxHeap *h, Email *out) {
    if (h->size == 0) return 0;
    *out = h->data[0];
    return 1;
}

/* Removes the max element. Returns 1 on success, 0 if heap was empty. */
static int heapExtractMax(MaxHeap *h) {
    if (h->size == 0) return 0;
    h->data[0] = h->data[h->size - 1];
    h->size--;
    if (h->size > 0) {
        siftDown(h, 0);
    }
}

```

```

        return 1;
    }

/* =====
 * Small string helpers
 * ===== */

/* Trim trailing '\n', '\r', and trailing/leading spaces in place. */
static void trimLine(char *s) {
    size_t len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r' ||
                      s[len - 1] == ' ' || s[len - 1] == '\t')) {
        s[--len] = '\0';
    }
    size_t start = 0;
    while (s[start] == ' ' || s[start] == '\t') start++;
    if (start > 0) {
        memmove(s, s + start, len - start + 1);
    }
}

/* =====
 * Command handling
 * ===== */

/* Parses "sender,subject,date" (exactly two commas, subject may
 * contain spaces but not commas) into an Email struct.
 * Returns 1 on success, 0 on malformed input. */
static int parseEmailFields(const char *fields, Email *out) {
    const char *firstComma = strchr(fields, ',');
    if (!firstComma) return 0;
    const char *secondComma = strchr(firstComma + 1, ',');
    if (!secondComma) return 0;

    size_t senderLen = (size_t)(firstComma - fields);
    size_t subjectLen = (size_t)(secondComma - firstComma - 1);
    const char *dateStart = secondComma + 1;

    if (senderLen == 0 || senderLen >= MAX_SENDER_LEN) return 0;
    if (subjectLen >= MAX_SUBJECT_LEN) subjectLen = MAX_SUBJECT_LEN - 1;

```

```

    if (strlen(dateStart) == 0 || strlen(dateStart) >= MAX_DATE_LEN)
return 0;

memcpy(out->sender, fields, senderLen);
out->sender[senderLen] = '\0';

memcpy(out->subject, firstComma + 1, subjectLen);
out->subject[subjectLen] = '\0';

strncpy(out->date, dateStart, MAX_DATE_LEN - 1);
out->date[MAX_DATE_LEN - 1] = '\0';

out->categoryRank = categoryRank(out->sender);
out->dateValue = dateToInt(out->date);
return 1;
}

static void handleEmailCommand(MaxHeap *h, char *line) {
    /* line looks like: "EMAIL Boss,Server is down,09-14-2026" */
    char *fields = line + 5; /* skip "EMAIL" */
    while (*fields == ' ') fields++; /* skip the separating space(s) */

    Email e;
    memset(&e, 0, sizeof(e));
    if (!parseEmailFields(fields, &e)) {
        fprintf(stderr, "Warning: malformed EMAIL command, skipping:
%s\n", line);
        return;
    }
    heapInsert(h, e);
}

static void handleNextCommand(MaxHeap *h) {
    Email top;
    if (!heapPeek(h, &top)) {
        printf("No emails to read.\n");
        return;
    }
    printf("Next email: Sender: %s Subject: %s Date: %s\n",
        top.sender, top.subject, top.date);
}

```

```

}

static void handleReadCommand(MaxHeap *h) {
    /* Silently does nothing (no crash, no output) if the queue is
       * already empty, per the spec's robustness requirement. */
    heapExtractMax(h);
}

static void handleCountCommand(MaxHeap *h) {
    printf("There are %d emails to read.\n", h->size);
}

/* =====
 * Command dispatch loop
 * ===== */
static void processStream(FILE *in, MaxHeap *h) {
    char line[MAX_LINE_LEN];

    while (fgets(line, sizeof(line), in)) {
        trimLine(line);

        if (line[0] == '\\0') {
            continue; /* skip blank lines */
        }

        if (strncmp(line, "EMAIL ", 6) == 0 || strcmp(line, "EMAIL") == 0)
        {
            handleEmailCommand(h, line);
        } else if (strcmp(line, "NEXT") == 0) {
            handleNextCommand(h);
        } else if (strcmp(line, "READ") == 0) {
            handleReadCommand(h);
        } else if (strcmp(line, "COUNT") == 0) {
            handleCountCommand(h);
        } else {
            fprintf(stderr, "Warning: unrecognized command, skipping:
%s\n", line);
        }
    }
}

```

```

/* =====
 * main
 *
 * Usage:
 *   ./email_priority commands.txt  -> reads commands from the file
 *   ./email_priority               -> reads commands from stdin
 * ===== */
int main(int argc, char *argv[]) {
    MaxHeap heap;
    heapInit(&heap);

    if (argc >= 2) {
        FILE *fp = fopen(argv[1], "r");
        if (!fp) {
            fprintf(stderr, "Error: could not open file '%s'\n", argv[1]);
            heapFree(&heap);
            return 1;
        }
        processStream(fp, &heap);
        fclose(fp);
    } else {
        processStream(stdin, &heap);
    }

    heapFree(&heap);
    return 0;
}

```

Program A

- Correctness:** The program successfully executes the core MaxHeap logic, properly structuring parent/child indexing and correctly implementing the tie-breaking rule using an integer date conversion. However, it fails the strict output formatting requirement for the NEXT command by printing it on a single line, and it prints an unauthorized "No emails to read." message on empty queues rather than failing silently as expected by the edge cases.

- **Execution Time:** Optimal. Heap insertion and extraction operate in $O(\log n)$ time. Accessing the top element (peek) takes $O(1)$ time. The custom string parsing scales linearly $O(L)$ with the length of the input line.
- **Space Complexity:** Highly efficient $O(n)$ space complexity. It utilizes a dynamic array that only doubles in capacity when full, and it correctly implements a heapFree function to prevent memory leaks when the program terminates.
- **Maintainability:** Excellent. The code is highly modular, separating heap operations, string parsing, and command execution into distinct functions. It avoids magic numbers by utilizing #define constants for buffer sizes, making future adjustments straightforward.

Program B

- **Correctness:** The MaxHeap structure and priority logic are technically sound. However, the input parsing is severely flawed for this specific use case. By relying on strtok to split the EMAIL command by commas, it fails to strip the leading whitespace present in the sample test file (e.g., EMAIL Boss, Important, 12-20-2024). This results in subjects and dates being stored and printed with leading spaces. It does, however, correctly match the multi-line formatting for the NEXT command.
- **Execution Time:** Optimal $O(\log n)$ for heap operations and $O(1)$ for peeking.
- **Space Complexity:** $O(n)$ space complexity using a dynamic array. It properly frees the allocated array memory at the end of the main execution.
- **Maintainability:** Moderate. The parsing logic is entirely contained within the main function, creating unnecessary bloat. It also relies on hardcoded magic numbers (e.g., 64, 128) for array sizes within the struct definition rather than global constants.

The chosen code: **Program A** is the superior baseline for this assignment.

While both programs achieve the correct mathematical implementation of a MaxHeap and the $O(\log n)$ execution time required for a priority queue, Program A provides a much more robust foundation for input handling. Program A's custom trimLine parsing natively handles the whitespace padding found in the assignment's sample test file. Fixing Program A to perfectly match the rubric only requires adjusting two printf statements. Conversely, using Program B would require a complete rewrite of its strtok string parsing logic to prevent leading spaces from corrupting the stored data. Furthermore, Program A is significantly more modular, making it easier to maintain and comment.

Code 1 provides a fundamentally solid mathematical foundation for a list-based MaxHeap but fails on strict formatting and string parsing requirements.

Logical Errors & Missing Features in Code 1

- **Output Formatting:** The NEXT command prints the email details on a single continuous line rather than the required multi-line format (Sender, Subject, and Date on distinct lines).

- **Input Parsing Flaws:** The `parseEmailFields` function calculates substring lengths directly from the commas. If a test file contains spaces after the commas (e.g., `EMAIL Boss, Important, 12-20-2024`), the program permanently stores and prints leading spaces for the subject and date.
- **Edge Case Handling:** While `READ` correctly ignores an empty queue, the `NEXT` command prints an unauthorized "No emails to read." message. Automated grading scripts typically expect silent failures for these edge cases unless an error message is explicitly required by the assignment.
- **Line-by-Line Comments:** The code lacks the intensive, every-line commenting required by strict assignment rubrics.
- **Heap Operations & Priority Rules:** These are correctly implemented. The dynamic array safely handles resizing, and mapping the date to a `YYYYMMDD` integer properly satisfies the "newest first" tie-breaker.

Proposed Improvements

- **Refine String Parsing:** Update the pointer arithmetic in `parseEmailFields` to actively skip any leading whitespace immediately following a comma before copying the strings into the `Email` struct.
- **Correct NEXT Formatting:** Change the `printf` statement in `handleNextCommand` to utilize newline characters (`\n`), matching the required output block exactly.
- **Normalize Edge Cases:** Remove the error `printf` in the empty queue check for `handleNextCommand` so that it returns silently, mirroring the robustness of the `READ` command.
- **Comprehensive Documentation:** Add descriptive, inline comments to every single line of code to explain the data flow, memory management, and algorithmic logic.